



《AI大模型Ragent项目》——Chat同步调用与模板方法

来自：拿个offer-开源&项目实战



马丁

2026年04月13日 09:33

上一篇讲了 `executeWithFallback` 如何遍历候选列表、逐个尝试、失败后切换到下一个候选。每个候选的实际调用就是一句 `client.chat(request, target)`。

那这一行代码背后到底发生了什么？一个 `ChatRequest` 对象是怎么变成一个发往百炼的 HTTP POST 请求的？百炼返回的 JSON 响应又是

怎么变成一个 `String` 返回给业务层的？HTTP 500 错误是怎么触发故障转移的？

这一篇打开 Chat 子系统的黑盒，看看从业务层调用到供应商 HTTP 请求之间的完整链路。

接口分层：LLMService 与 ChatClient

1. 两层接口的设计

项目里和 Chat 相关的接口有两层。第一层是业务层使用的 `LLMService`：

```
public interface LLMService {

    default String chat(String prompt) {
        ChatRequest req = ChatRequest.builder()
            .messages(List.of(ChatMessage.user(prompt)))
            .build();
        return chat(req);
    }

    String chat(ChatRequest request);

    default String chat(ChatRequest request, String modelId) {
        return chat(request);
    }

    default StreamCancellationHandle streamChat(String prompt, StreamCallback callback) {
        ChatRequest req = ChatRequest.builder()
            .messages(List.of(ChatMessage.user(prompt)))
            .build();
        return streamChat(req, callback);
    }

    StreamCancellationHandle streamChat(ChatRequest request, StreamCallback callback);
}
```

`LLMService` 面向业务层，接口里看不到任何 infra 概念——没有 `ModelTarget`，没有 `ProviderConfig`，没有供应商名称。业务代码只需要传一个 `ChatRequest`，就能拿到模型的回答。至于背后调的是百炼还是硅基流动，哪个模型先试哪个后试，业务层不需要关心。

第二层是供应商级别的 `ChatClient`：

```
public interface ChatClient {

    String provider();

    String chat(ChatRequest request, ModelTarget target);

    StreamCancellationHandle streamChat(ChatRequest request, StreamCallback callback, ModelTarget
}
```

`ChatClient` 面向供应商层，多了两个东西：`provider()` 返回供应商标识（比如 "bailian"、"siliconflow"、"ollama"），`chat` 方法多了一个 `ModelTarget` 参数——它包含了这次调用需要的所有运行时信息：模型名称、供应商 URL、API Key。

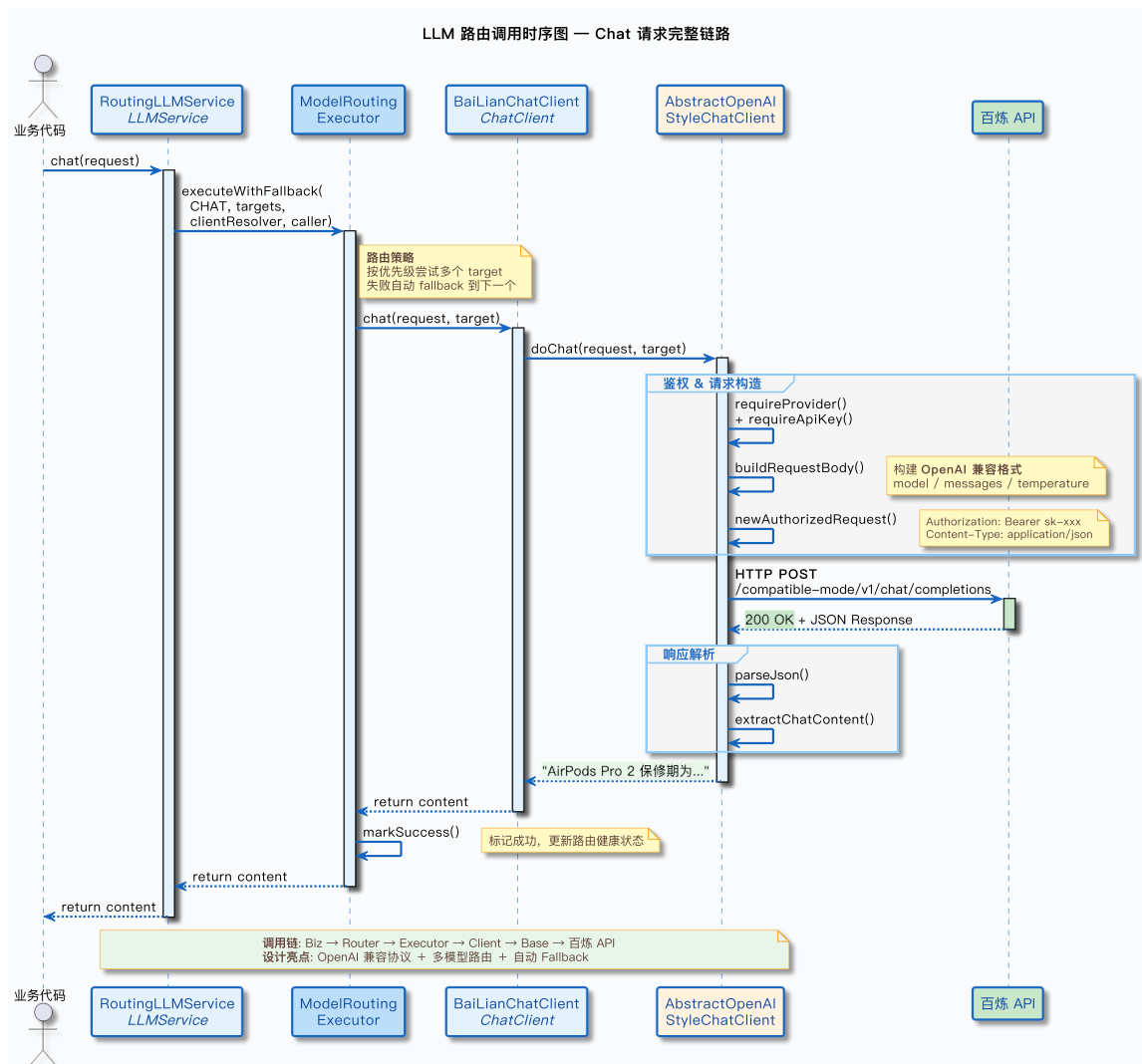
两层之间的桥梁是 RoutingLLMService。它实现了 LLMService 接口，内部使用 ModelSelector + ModelRoutingExecutor + ChatClient 完成路由和调用：

```
@Override
public String chat(ChatRequest request) {
    return executor.executeWithFallback(
        ModelCapability.CHAT,
        selector.selectChatCandidates(Boolean.TRUE.equals(request.getThinking())),
        target -> clientsByProvider.get(target.candidate().getProvider()),
        (client, target) -> client.chat(request, target)
    );
}
```

业务层调 LLMService.chat(request)，RoutingLLMService 通过 executeWithFallback 遍历候选列表，对每个候选查找对应的 ChatClient 实例，然后调 client.chat(request, target)。业务层完全不知道 ChatClient 的存在。

2. 调用链路图

从业务代码到供应商 HTTP 请求的完整路径：



整条链路分三段：**路由段** (RoutingLLMService → executeWithFallback) 负责选谁、怎么切换；**协议段** (AbstractOpenAIStyleChatClient.doChat) 负责构建 HTTP 请求和解析响应；**传输段** (OkHttp → 供应商 API) 负责发送和接收。

请求与响应数据结构

1. ChatRequest 与 ChatMessage

在看 doChat 的实现之前，先了解它的输入。ChatRequest 定义在 framework 模块，是一个统一的大模型请求对象：

```
@Data
@Builder
public class ChatRequest {

    @Default
    private List<ChatMessage> messages = new ArrayList<>();

    private Double temperature;
    private Double topP;
    private Integer topK;
    private Integer maxTokens;
    private Boolean thinking;
    private Boolean enableTools;
}
```

messages 是消息列表，每条消息由 ChatMessage 表示：

```
@Data
public class ChatMessage {

    public enum Role {
        SYSTEM,
        USER,
        ASSISTANT;
    }

    private Role role;
    private String content;
    private String thinkingContent;
    private Integer thinkingDuration;

    public static ChatMessage system(String content) {
        return new ChatMessage(Role.SYSTEM, content);
    }

    public static ChatMessage user(String content) {
        return new ChatMessage(Role.USER, content);
    }

    public static ChatMessage assistant(String content) {
        return new ChatMessage(Role.ASSISTANT, content);
    }
}
```

三个静态工厂方法让构建消息很简洁。比如一次带 System Prompt 的问答：

```
ChatRequest request = ChatRequest.builder()
    .messages(List.of(
        ChatMessage.system("你是一个电商客服助手，请根据提供的知识回答用户问题。"),
        ChatMessage.user("AirPods Pro 2 的保修期是多久？")
    ))
    .temperature(0.1)
    .build();
```

2. 到 OpenAI 格式的映射

上面的 ChatRequest 会被 buildRequestBody 方法转换成 OpenAI Chat Completions API 的 JSON 格式。映射关系是直接的：

请求体：

```
{
  "model": "qwen3-max",
  "messages": [
    {"role": "system", "content": "你是一个电商客服助手，请根据提供的知识回答用户问题。"},
    {"role": "user", "content": "AirPods Pro 2 的保修期是多久?" }
  ],
  "temperature": 0.1
}
```

model 不是从 ChatRequest 里来的——它来自 ModelTarget.candidate().getModel(), 也就是 YAML 配置中的模型名。topP、topK、maxTokens 为 null 时不加进请求体，由供应商使用默认值。

响应体：

```
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "AirPods Pro 2 的保修期为 1 年有限保修..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 42,
    "completion_tokens": 56,
    "total_tokens": 98
  }
}
```

extractChatContent 方法从响应体中提取 choices[0].message.content——就是模型的回答文本。

模板方法：AbstractOpenAIStyleChatClient

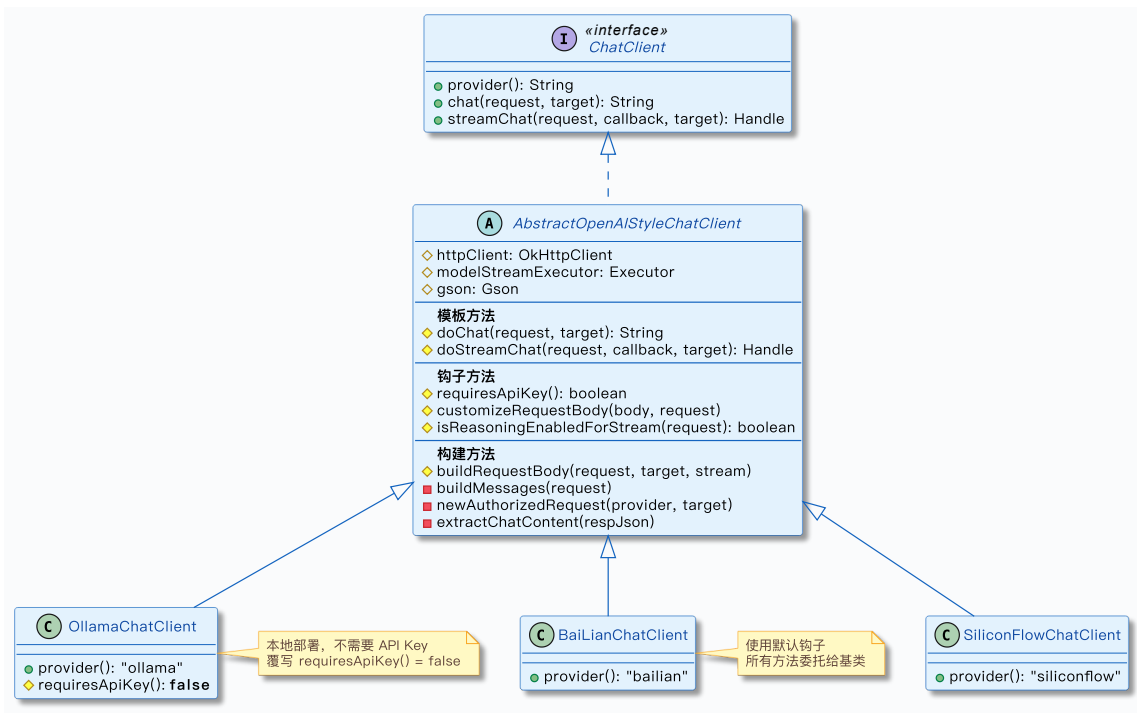
1. 为什么用模板方法

项目目前支持三个供应商：百炼（阿里云）、硅基流动、Ollama。这三家都兼容 OpenAI Chat Completions API——请求格式一样（model + messages + 参数），响应格式一样（choices[0].message.content），端点路径也类似（/v1/chat/completions 或变体）。

如果每个供应商都从头实现一遍 HTTP 请求构建、JSON 序列化、响应解析、错误处理，90% 的代码是重复的。差异就那么一点：Ollama 不需要 API Key，百炼和硅基流动需要；个别供应商可能有特殊的请求字段；URL 端点不一样。

模板方法模式解决这个问题：把相同的 90%（请求构建、HTTP 调用、响应解析、错误处理）放在抽象基类 AbstractOpenAIStyleChatClient 里，把不同的 10%（认证方式、特殊字段）留给子类通过钩子方法覆写。

2. 类继承结构



三个子类的代码量极少（各约 30 行），所有协议处理逻辑都在基类里。

3. 三个钩子方法

基类定义了三个可覆写的钩子方法：

```

protected boolean requiresApiKey() {
    return true;
}

protected void customizeRequestBody(JsonObject body, ChatRequest request) {
    if (Boolean.TRUE.equals(request.getThinking())) {
        body.addProperty("enable_thinking", true);
    }
}

protected boolean isReasoningEnabledForStream(ChatRequest request) {
    return Boolean.TRUE.equals(request.getThinking());
}

```

`requiresApiKey()`——控制是否校验和携带 API Key。默认返回 `true`。Ollama 是本地部署的推理服务，没有云端认证机制，所以 `OllamaChatClient` 覆写为 `false`。覆写后，`doChat` 会跳过 API Key 校验，`newAuthorizedRequest` 也不会添加 `Authorization` 头。

`customizeRequestBody(JsonObject, ChatRequest)`——向请求体添加供应商特有字段。默认实现在 `thinking = true` 时添加 `"enable_thinking": true`，这是百炼和部分兼容供应商支持的深度思考开关。如果某个供应商有自己独特的字段（比如特殊的采样参数），子类可以覆写这个方法添加。

`isReasoningEnabledForStream(ChatRequest)`——流式调用时是否解析 `reasoning_content` 字段。本篇不展开，后续流式解析的文章会讲。

4. doChat 模板方法完整代码

`doChat` 是同步调用的核心，定义了从校验到返回结果的完整骨架：

```

protected String doChat(ChatRequest request, ModelTarget target) {
    AIModelProperties.ProviderConfig provider = HttpResponseHelper.requireProvider(target, provide
    if (requiresApiKey()) {
        HttpResponseHelper.requireApiKey(provider, provider());
    }

    JsonObject reqBody = buildRequestBody(request, target, false);
    Request requestHttp = newAuthorizedRequest(provider, target)
        .post(RequestBody.create(reqBody.toString(), HttpMediaTypes.JSON))

```

```

        .build();

JsonObject respJson;
try (Response response = httpClient.newCall(requestHttp).execute()) {
    if (!response.isSuccessful()) {
        String body = HttpResponseHelper.readBody(response.body());
        log.warn("{} 同步请求失败: status={}, body={}", provider(), response.code(), body);
        throw new ModelClientException(
            provider() + " 同步请求失败: HTTP " + response.code(),
            ModelClientErrorType.fromHttpStatus(response.code()),
            response.code()
        );
    }
    respJson = HttpResponseHelper.parseJson(response.body(), provider());
} catch (IOException e) {
    throw new ModelClientException(
        provider() + " 同步请求失败: " + e.getMessage(),
        ModelClientErrorType.NETWORK_ERROR, null, e);
}

return extractChatContent(respJson);
}

```

按执行顺序拆解：

1. 校验供应商配置：requireProvider 确保 ModelTarget 里有供应商配置（ProviderConfig 非空）。
2. 校验 API Key：如果 requiresApiKey() 为 true，检查供应商配置中的 apiKey 字段非空。Ollama 跳过这步。
3. 构建请求体：buildRequestBody 把 ChatRequest 转换为 OpenAI 格式的 JSON。stream = false 表示同步调用。
4. 构建 HTTP 请求：newAuthorizedRequest 解析 URL 并添加 Authorization 头，然后设置 POST body。
5. 发送请求：httpClient.newCall(requestHttp).execute() 同步发送 HTTP 请求。用 try-with-resources 确保响应体关闭。
6. 错误检查：非 2xx 响应时，读取响应体作为错误信息，按 HTTP 状态码分类错误类型，抛出 ModelClientException。
7. 解析响应：parseJson 把响应体解析为 JsonObject。
8. 提取内容：extractChatContent 从 choices[0].message.content 提取模型回答。
9. 网络异常：IOException 被包装为 ModelClientException(NETWORK_ERROR)。

5. buildRequestBody——请求体构建

```

protected JsonObject buildRequestBody(ChatRequest request, ModelTarget target, boolean stream) {
    JsonObject body = new JsonObject();
    body.addProperty("model", HttpResponseHelper.requireModel(target, provider()));
    if (stream) {
        body.addProperty("stream", true);
    }

    body.add("messages", buildMessages(request));

    if (request.getTemperature() != null) {
        body.addProperty("temperature", request.getTemperature());
    }
    if (request.getTopP() != null) {
        body.addProperty("top_p", request.getTopP());
    }
    if (request.getTopK() != null) {
        body.addProperty("top_k", request.getTopK());
    }
    if (request.getMaxTokens() != null) {
        body.addProperty("max_tokens", request.getMaxTokens());
    }
}

```

```
customizeRequestBody(body, request);
return body;
}
```

model 字段来自 ModelTarget——也就是 YAML 配置中的候选模型名（比如 qwen3-max、qwen-plus-latest）。生成参数（temperature / top_p / top_k / max_tokens）只在非 null 时添加，null 则不出现在请求体中，由供应商使用默认值。

最后一行 customizeRequestBody(body, request) 是钩子调用点——子类可以在这里添加供应商特有字段。默认实现在 thinking = true 时添加 "enable_thinking": true。

buildMessages 方法把 List<ChatMessage> 转换为 OpenAI 格式的 messages JSON 数组：

```
private JSONArray buildMessages(ChatRequest request) {
    JSONArray arr = new JSONArray();
    List<ChatMessage> messages = request.getMessages();
    if (CollUtil.isEmpty(messages)) {
        for (ChatMessage m : messages) {
            JsonObject msg = new JsonObject();
            msg.addProperty("role", toOpenAiRole(m.getRole()));
            msg.addProperty("content", m.getContent());
            arr.add(msg);
        }
    }
    return arr;
}

private String toOpenAiRole(ChatMessage.Role role) {
    return switch (role) {
        case SYSTEM -> "system";
        case USER -> "user";
        case ASSISTANT -> "assistant";
    };
}
```

映射很直接：SYSTEM → "system"、USER → "user"、ASSISTANT → "assistant"。Java 17 的 switch 表达式让代码很简洁。

6. newAuthorizedRequest——URL 解析与认证

```
private Request.Builder newAuthorizedRequest(
    AIModelProperties.ProviderConfig provider, ModelTarget target) {
    Request.Builder builder = new Request.Builder()
        .url(ModelUrlResolver.resolveUrl(provider, target.candidate(), ModelCapability.CHAT));
    if (requiresApiKey()) {
        builder.addHeader("Authorization", "Bearer " + provider.getApiKey());
    }
    return builder;
}
```

两件事。第一，URL 通过 ModelUrlResolver.resolveUrl() 解析——第二篇讲过的两级优先级：候选有自己的 url 就直接用，没有就用供应商的 url + endpoints.chat 拼接。

第二，如果 requiresApiKey() 为 true，添加 Authorization: Bearer <apiKey> 头。百炼和硅基流动都需要，Ollama 不需要。requiresApiKey() 返回 false 时，这一行直接跳过。

7. extractChatContent——响应解析

```
private String extractChatContent(JsonObject respJson) {
    if (respJson == null || !respJson.has("choices")) {
        throw new ModelClientException(
            provider() + " 响应缺少 choices", ModelClientErrorType.INVALID_RESPONSE, null);
    }
}
```

```
JSONArray choices = respJson.getAsJSONArray("choices");
if (choices == null || choices.isEmpty()) {
    throw new ModelClientException(
        provider() + " 响应 choices 为空", ModelClientErrorType.INVALID_RESPONSE, null);
}
JsonObject choice0 = choices.get(0).getAsJsonObject();
if (choice0 == null || !choice0.has("message")) {
    throw new ModelClientException(
        provider() + " 响应缺少 message", ModelClientErrorType.INVALID_RESPONSE, null);
}
JsonObject message = choice0.getAsJsonObject("message");
if (message == null || !message.has("content") || message.get("content").isJsonNull()) {
    throw new ModelClientException(
        provider() + " 响应缺少 content", ModelClientErrorType.INVALID_RESPONSE, null);
}
return message.get("content").getString();
}
```

逐层校验，每一步失败都抛 `ModelClientException(INVALID_RESPONSE)`。解析路径：`respJson` → `choices`（数组）→ `choices[0]`（对象）→ `message`（对象）→ `content`（字符串）。

为什么要这么多 `null` 检查？因为大模型供应商的响应不是总能信任的。有时候供应商返回格式不完全符合 `OpenAI` 规范（比如某些边缘情况下 `content` 为 `null`），有时候网络问题导致响应被截断。逐层校验加上明确的错误信息，排查问题时能快速定位到具体是哪一层出了问题。

HTTP 错误处理体系

1. ModelClientErrorType——错误分类

```
public enum ModelClientErrorType {

    UNAUTHORIZED,
    RATE_LIMITED,
    SERVER_ERROR,
    CLIENT_ERROR,
    NETWORK_ERROR,
    INVALID_RESPONSE,
    PROVIDER_ERROR;

    public static ModelClientErrorType fromHttpStatus(int status) {
        if (status == 401 || status == 403) {
            return UNAUTHORIZED;
        }
        if (status == 429) {
            return RATE_LIMITED;
        }
        if (status >= 500) {
            return SERVER_ERROR;
        }
        return CLIENT_ERROR;
    }
}
```

七种错误类型，其中前四种可以从 HTTP 状态码自动推断：

HTTP 状态码	错误类型	含义	典型场景
401 / 403	UNAUTHORIZED	认证失败	API Key 过期或无效
429	RATE_LIMITED	频率超限	供应商 QPS 限流
500+	SERVER_ERROR	服务端错误	供应商平台故障
其他 4xx	CLIENT_ERROR	客户端错误	请求参数不合法

另外三种不来自 HTTP 状态码：NETWORK_ERROR 对应 IOException（网络超时、连接中断）；INVALID_RESPONSE 对应响应格式异常（JSON 解析失败、缺少字段）；PROVIDER_ERROR 作为通用兜底。

2. ModelClientException——结构化异常

```
@Getter
public class ModelClientException extends RuntimeException {

    private final ModelClientErrorType errorType;
    private final Integer statusCode;

    public ModelClientException(String message, ModelClientErrorType errorType,
                                Integer statusCode, Throwable cause) {
        super(message, cause);
        this.errorType = errorType;
        this.statusCode = statusCode;
    }

    public ModelClientException(String message, ModelClientErrorType errorType,
                                Integer statusCode) {
        super(message);
        this.errorType = errorType;
        this.statusCode = statusCode;
    }
}
```

ModelClientException 是 RuntimeException 子类，携带两个结构化字段：errorType（错误分类）和 statusCode（HTTP 状态码，网络错误时为 null）。

它是 RuntimeException 而不是受检异常，这很重要——ModelCaller<C, T> 的 call 方法声明了 throws Exception，doChat 抛出的 ModelClientException 会直接传递到 executeWithFallback 的 catch 块，触发 markFailure + 故障转移。整个错误传递链不需要显式声明。

3. HttpResponseHelper——校验工具

```
@NoArgsConstructor(access = lombok.AccessLevel.PRIVATE)
public final class HttpResponseHelper {

    private static final Gson GSON = new Gson();

    public static String readBody(ResponseBody body) throws IOException {
        if (body == null) {
            return "";
        }
        return new String(body.bytes(), StandardCharsets.UTF_8);
    }

    public static JsonObject parseJson(ResponseBody body, String label) throws IOException {
        if (body == null) {
            throw new ModelClientException(
                label + " 响应为空", ModelClientErrorType.INVALID_RESPONSE, null);
        }
        String content = body.string();
        return GSON.fromJson(content, JsonObject.class);
    }

    public static AIModelProperties.ProviderConfig requireProvider(
        ModelTarget target, String label) {
        if (target == null || target.provider() == null) {
            throw new IllegalStateException(label + " 提供商配置缺失");
        }
        return target.provider();
    }
}
```

```
public static void requireApiKey(AIModelProperties.ProviderConfig provider, String label) {
    if (provider.getApiKey() == null || provider.getApiKey().isBlank()) {
        throw new IllegalStateException(label + " API密钥缺失");
    }
}

public static String requireModel(ModelTarget target, String label) {
    if (target == null || target.candidate() == null
        || target.candidate().getModel() == null) {
        throw new IllegalStateException(label + " 模型名称缺失");
    }
    return target.candidate().getModel();
}
}
```

四个 `require*` 方法做前置校验，`parseJson` 做响应解析。它们被 `doChat` 在不同步骤中调用，统一了校验逻辑——不管是百炼还是硅基流动，校验代码都一样，不需要每个客户端重复写。

`readBody` 用于错误场景下读取响应体（供应商通常在非 200 响应的 body 里返回错误详情），`parseJson` 用于正常场景下解析 JSON 响应。

4. 错误处理与故障转移的联动

把错误处理和上一篇的故障转移串起来，走一个完整的场景：

用户问：“AirPods Pro 2 的保修期是多久？”候选列表是 `qwen3-max` → `glm-4.7` → `qwen-plus` → `qwen3-local`。

1. `executeWithFallback` 遍历到 `qwen3-max`，调用 `BailianChatClient.chat(request, target)`
2. `doChat` 内部构建 HTTP 请求，发往百炼 API
3. 百炼返回 **HTTP 500**（内部服务器错误）
4. `doChat` 检测到 `!response.isSuccessful()`，读取错误响应体
5. `ModelClientErrorType.fromHttpStatus(500)` 返回 `SERVER_ERROR`
6. 抛出 `new ModelClientException("bailian 同步请求失败: HTTP 500", SERVER_ERROR, 500)`
7. 异常传递到 `executeWithFallback` 的 `catch` 块
8. `healthStore.markFailure("qwen3-max")` → `consecutiveFailures = 1`
9. 记录日志: "Chat model failed, fallback to next. modelId=qwen3-max, provider=bailian"
10. 继续下一个候选 `glm-4.7` (`SiliconFlowChatClient`)，调用成功，返回结果

错误从 HTTP 层（状态码 500）→ 协议层（`ModelClientException`）→ 路由层（`executeWithFallback` catch）→ 熔断层（`markFailure`），层层传递，最终触发故障转移。业务层拿到的是 `glm-4.7` 的正确回答，完全感知不到 `qwen3-max` 的故障。

三个供应商客户端

1. OllamaChatClient

```
@Slf4j
@Service
public class OllamaChatClient extends AbstractOpenAISTyleChatClient {

    public OllamaChatClient(OkHttpClient httpClient, Executor modelStreamExecutor) {
        super(httpClient, modelStreamExecutor);
    }

    @Override
    public String provider() {
        return ModelProvider.OLLAMA.getId();
    }

    @Override
    protected boolean requiresApiKey() {
        return false;
    }
}
```

```

    }

    @Override
    @RagTraceNode(name = "ollama-chat", type = "LLM_PROVIDER")
    public String chat(ChatRequest request, ModelTarget target) {
        return doChat(request, target);
    }

    @Override
    @RagTraceNode(name = "ollama-stream-chat", type = "LLM_PROVIDER")
    public StreamCancellationHandle streamChat(
        ChatRequest request, StreamCallback callback, ModelTarget target) {
        return doStreamChat(request, callback, target);
    }
}

```

唯一的差异：requiresApiKey() 返回 false。Ollama 是本地部署的推理服务，运行在 localhost:11434，没有云端认证。chat 和 streamChat 直接委托给基类的 doChat / doStreamChat，自己不加任何逻辑。

@RagTraceNode 注解用于链路追踪——记录这次调用走的是哪个供应商、耗时多少。加在子类而不是基类上，是因为追踪需要区分供应商，而基类是抽象的，无法标注具体的供应商名。

2. BaiLianChatClient 和 SiliconFlowChatClient

```

@Slf4j
@Service
public class BaiLianChatClient extends AbstractOpenAISTyleChatClient {

    public BaiLianChatClient(OkHttpClient httpClient, Executor modelStreamExecutor) {
        super(httpClient, modelStreamExecutor);
    }

    @Override
    public String provider() {
        return ModelProvider.BAI_LIAN.getId();
    }

    @Override
    @RagTraceNode(name = "bailian-chat", type = "LLM_PROVIDER")
    public String chat(ChatRequest request, ModelTarget target) {
        return doChat(request, target);
    }

    @Override
    @RagTraceNode(name = "bailian-stream-chat", type = "LLM_PROVIDER")
    public StreamCancellationHandle streamChat(
        ChatRequest request, StreamCallback callback, ModelTarget target) {
        return doStreamChat(request, callback, target);
    }
}

```

```

@Slf4j
@Service
public class SiliconFlowChatClient extends AbstractOpenAISTyleChatClient {

    public SiliconFlowChatClient(OkHttpClient httpClient, Executor modelStreamExecutor) {
        super(httpClient, modelStreamExecutor);
    }

    @Override
    public String provider() {
        return ModelProvider.SILICON_FLOW.getId();
    }
}

```

```
    }

    @Override
    @RagTraceNode(name = "siliconflow-chat", type = "LLM_PROVIDER")
    public String chat(ChatRequest request, ModelTarget target) {
        return doChat(request, target);
    }

    @Override
    @RagTraceNode(name = "siliconflow-stream-chat", type = "LLM_PROVIDER")
    public StreamCancellationHandle streamChat(
        ChatRequest request, StreamCallback callback, ModelTarget target) {
        return doStreamChat(request, callback, target);
    }
}
```

两个类几乎一模一样——连钩子方法都没有覆写，全用基类的默认值。差异只有 provider() 返回的字符串不同。百炼和硅基流动都是公有云服务，都需要 API Key 认证，都兼容 OpenAI 协议，连深度思考的 enable_thinking 字段都一样。

3. 差异对比

供应商	provider()	requiresApiKey	customizeRequestBody	URL 端点示例
Ollama	"ollama"	false	默认 (enable_thinking)	http://localhost:11434/v1/chat/completions
百炼	"bailian"	true (默认)	默认 (enable_thinking)	https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions
硅基流动	"siliconflow"	true (默认)	默认 (enable_thinking)	https://api.siliconflow.cn/v1/chat/completions

三个供应商客户端之所以这么简洁，是因为 OpenAI 兼容协议的标准化做得足够好。协议层面的差异被抽象基类完全吸收了，供应商层面只剩下认证方式这一个差异点。

4. 如何新增一个供应商

如果你要接入一个新的 OpenAI 兼容供应商（比如 DeepSeek API），步骤是这样的：

- 1. 在 ModelProvider 枚举中添加新供应商：DEEP_SEEK("deepseek")
- 2. 创建客户端类，继承 AbstractOpenAIStyleChatClient
- 3. 实现 provider(): 返回 ModelProvider.DEEP_SEEK.getId()
- 4. 按需覆写钩子方法：不需要 API Key? 覆写 requiresApiKey()。有特殊请求字段? 覆写 customizeRequestBody()
- 5. 加 @Service 注解：注册为 Spring Bean，Spring 会自动把它收集到 List<ChatClient> 中
- 6. 在 YAML 配置中添加供应商配置和候选配置：providers.deepseek 节点配 URL 和 API Key，chat.candidates 里加一个候选

如果供应商不兼容 OpenAI 协议（比如用的是完全不同的请求格式），就不能继承 AbstractOpenAIStyleChatClient——需要直接实现 ChatClient 接口，自己处理请求构建和响应解析。但只要实现了 ChatClient 接口并注册为 Spring Bean，RoutingLLMService 的路由和故障转移机制就能自动适配，不需要改任何路由层代码。

小结与下一步

回顾这一篇的核心要点：

- LLMService 面向业务层，ChatClient 面向供应商层，两层接口隔离了业务逻辑和 infra 细节
- AbstractOpenAIStyleChatClient 用模板方法封装了 OpenAI 兼容协议的请求构建、HTTP 调用、响应解析的完整骨架
- 三个钩子方法（requiresApiKey、customizeRequestBody、isReasoningEnabledForStream）让子类用最少的代码表达供应商差异
- 三个供应商客户端代码极其简洁（各约 30 行），因为 OpenAI 兼容协议足够标准化

HTTP 错误处理体系（ModelClientErrorType 分类 + ModelClientException 结构化异常）和 executeWithFallback 无缝联动，错误从 HTTP 层传递到路由层触发故障转移

新增一个 OpenAI 兼容供应商只需要：枚举 → 继承 → provider() → 钩子覆写 → @Service → YAML 配置

不过本篇只讲了同步调用路径。AbstractOpenAIStyleChatClient 还有一个 doStreamChat 模板方法——它的实现更复杂：OkHttp 的响应要逐行读取、SSE 协议要按 data: 前缀解析、[DONE] 终止标记要识别、流式内容要通过 StreamCallback 异步推送、调用取消要通过 StreamCancellationHandle 处理。

下一篇进入 **SSE 流式解析与异步执行**——深入 doStreamChat 的底层实现：OpenAIStyleSseParser 如何解析 SSE 协议的 data: 行和 delta.content / delta.reasoning_content 字段，StreamAsyncExecutor 如何将 OkHttp 的阻塞式流式读取提交到专用线程池异步执行，以及 StreamCancellationHandle 的取消机制如何保证资源正确释放。

